

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(ДИПЛОМ)**

по направлению подготовки

09.03.01 – «Информатика и вычислительная техника»

**GRADUATION THESIS
(BACHELORS GRADUATE THESIS)**

Field of Study

09.03.01 – “Computer Science”

Направленность (профиль) образовательной программы

«Анализ данных и искусственный интеллект»

Area of Specialization / Academic Program Title:

“Data Analysis and Artificial Intelligence”

**Тема /
Topic**

**Генеративная адаптация веб-интерфейсов под цели
пользователя в
режиме реального времени / User-Aligned Web Interfaces:
Just-in-Time Generative Adaptation of Existing UIs**

Работу выполнил /
Thesis is executed by

**Левада Андрей
Романович / todo**

подпись / signature

Руководитель
выпускной
квалификационной
работы / Graduation
Thesis Supervisor

todo / todo

подпись / signature

Contents

1	Introduction	7
1.1	Background	7
1.2	Key contributions	8
2	Related work	9
2.1	Generative Adaptation of UI	9
2.2	Generation of UI	9
2.3	Adaptation of UI	9
3	Methods	10
3.1	Design Goals	10
3.2	Key Challenges	11
3.2.1	Interaction model	11
3.2.2	Representation model	12
3.3	System Design Summary	14
3.4	User Testing Protocol	14
3.5	AI Usage Disclosure	16
4	Implementation	17

4.1 Overview	17
4.2 Agentic System	18
4.2.1 Action Space	18
4.2.2 Workflow	19
4.2.3 Proprietary model aside	20
4.3 DOM Compression	20
4.3.1 Visibility filtering	21
4.3.2 Lossy structural map	21
4.4 Browser Extension Wrapper (Fiber)	22
4.5 Security Concerns	23
5 Results	24
5.1 Browser Extension	24
5.2 User Testing	24
5.3 DOM Compression	24
6 Discussion	26
6.1 Emergent Behaviors	26
7 Conclusion	27
7.1 Limitations	27
Bibliography cited	28
A Source Code	29

List of Figures

4.1 Internet Shaper's system architecture	17
---	----

List of Tables

3.1	Measurements of sample DOMs before the compression	13
5.1	Character and token counts across 75 snapshots	24

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamur animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Chapter 1

Introduction

1.1 Background

Gray and dark patterns systematically misalign system and user goals. This happens when business and user goal diverge, and interface ends up being less helpful and more intrusive. We call such interfaces hostile. (**WIP**: citation needed here – not a full lit review, because this is not an actual point we study, but instead a motivation for the research to be done, the reason why it matters)

(**WIP**: in what way are they usually misaligned? categorization here, or is it too much out of focus? – bench requires categorization)

Prior HCI research shows that user interfaces can be made adaptive, generative, and even malleable by end users. (**WIP**: citation needed here for general overview of adaptive or customizable interfaces; the generative one will be thoroughly explored in the related works section)

However, we lack a conceptual and empirical example of how user-controlled adaptations of existing interfaces can function as a response to gray patterns.

This motivates the *Research Question* — How can adaptive generative interfaces counteract hostile UIs by re-aligning the interface with a user’s articulated goal?

We propose a *Solution* — a browser extension that uses large language models (LLMs) to adapt existing web user interfaces to a user-defined goal. This prototype takes a user request in a form of prompt as an input. It then generates edits to be applied to the currently opened web page. These edits get applied to the web page every session the user opens it.

1.2 Key contributions

1. **Internet Shaper** prototype — first of its kind proof of concept of a generative UI adaptation
2. **Fiber**, a browser extension framework, also used by the prototype.
3. Rule-based approach for LLM-to-webpage interactions.
4. Lossy and lossless DOM compression algorithms used by the prototype.

We also conduct a proof-of-concept user testing with $N=?$, showing feasibility of the interface adaptation in re-aligning user interfaces.

Chapter 2

Related work

We explore similar solutions that demonstrate UI generation and adaptation, from which a gap becomes apparent: very few interface adaptation methods exist.

2.1 Generative Adaptation of UI

2.2 Generation of UI

2.3 Adaptation of UI

Nearly all existing UI adaptation solutions require implementation by the application itself or its developers, with renders them unapplicable for our user-initiated setting. This section covers only adaptation methods that are controlled by the end user, of which few exist.

Chapter 3

Methods

The system is defined by the initial design constraints. We describe them and show our reasoning up to system design in this section. After that we describe the protocol of a proof-of-concept user study.

3.1 Design Goals

Unlike most adaptive interface methods — which are built into the application by its developers — this system operates on software that already exists and was not designed to be modified. The constraints reflect this:

- The system must have a way to change and override the existing application’s user interface. This key constraint motivated the browser extension as a host for the solution. Browser extensions can uniquely modify any opened website, making them well-suited for testing whether user-initiated interface adaptation is a feasible and useful approach to improving the user experience in hostile interfaces.
- The underlying LLM must have access to all the information visible to the user when the user edits a page.
- The changes must be persistent across sessions.
- Ideally, the system should be multi-turn to enable recovery from failed requests and tuning of results by the user.

These goals motivate the use of a browser extension environment, mainly because of its direct access to the Document Object Model (DOM) with unrestricted manipulation capabilities.

3.2 Key Challenges

These goals surface two interconnected challenges:

1. Interaction model: how can an LLM change the web page to re-align it?
2. Representation model: how do we represent the web page data to the LLM?

The two are tightly coupled — the representation must support whatever operations the interaction model requires.

3.2.1 Interaction model

The browser extension environment gives us unique access to the page's DOM and runtime context. From it, it is possible to change the DOM — modifying existing elements' structure, style, and events. That appears to be the only feasible way to reshape the environment: give the LLM a way to modify the DOM.

To achieve the persistence design goal, the LLM can generate idempotent change instructions to be applied on every page reload. For the exact format of these changes, several options exist:

1. Pseudo-language transformation instructions.
2. Generation of CSS styles.
3. Generation of JavaScript code.

The second option covers only a partial set of problems: the agent can hide elements, change their appearance, or transform existing layouts — but it cannot create new elements and flows, move existing elements, or modify logic. A few examples that motivated our choice:

Example. “Show only videos that are longer than 40 min” run on a playlist page on youtube.com. To successfully process this request, the persistent change must instruct the system to retrieve length data for each video — either from the page’s runtime variables or from displayed text in the DOM — and then perform a conditional check.

Frontier models excel at code generation, especially for tech stacks that are extensively present in their training data. Rather than creating a pseudo-language to handle these persistent changes, using JavaScript to model them is expected to yield better performance.

3.2.2 Representation model

There is a relevant adjacent area of research from which this work draws valuable insights: web agents — systems that can comprehend web pages and take actions on behalf of users. Specifically, we are interested in the methods web agents use to comprehend web pages.

Following [1]’s classification of comprehension techniques into text-based (which operates on code rather than natural text), screenshot-based, and multimodal — all approaches have shown promising results. However, since the interaction model relies on the DOM structure to query and modify components, the agent requires textual input, making screenshot-based perception infeasible. This paper focuses on text-based perception; multimodal perception is left for future work.

Some web-agent work relies on the accessibility tree rather than the DOM. The accessibility tree is the browser’s parallel representation of a page for assistive technologies such as screen readers. It preserves semantic structure while intentionally stripping away information that is purely visual. This makes it useful for many comprehension tasks, but it is not sufficient for our setting. We expect users to ask for visual adaptations such as changing layout, or hiding visual distractions. The system

therefore needs both semantic and visual context, so it uses DOM snapshots rather than the accessibility tree as its primary representation.

Existing approaches to DOM compacting also include pruning only task-relevant elements. For example, Prune4Web [2] uses programmatic filtering to reduce the candidate DOM elements. Such pruning is highly relevant to future versions of this system, especially when the requested change is localized to a small part of the page. In our case, however, many plausible user requests are not localized: restyling the page, changing the layout of content, or suppressing a broad class of distractions can make a large fraction of the DOM relevant. We therefore do not explore task-specific DOM pruning in this thesis. Instead, we focus on preserving broad page context: hierarchical relationships, visible content, and the overall tree structure.

The primary challenge with textual representation is the size of DOMs. To demonstrate, three unprocessed DOM snapshots were collected from logged-in websites — Instagram, Substack, and YouTube. These DOMs differ in file size, complexity, and underlying technologies:

TABLE 3.1
Sample DOM sizes

Instance	Snapshot size
Instagram	1,025,897 tokens, 1,964,595 chars
Substack	192,875 tokens, 453,795 chars
YouTube	1,629,793 tokens, 4,040,550 chars

Each DOM’s size was measured in tokens using Anthropic’s token measurement API. The context windows of the most advanced LLMs are mostly under one million tokens, with budget models typically around 200k. Passing the full DOM to the model as-is is not a viable baseline, as some DOMs will not fit in the context window. DOM compression is therefore necessary both to fit some DOMs inside the context window and to reduce noise with no semantic meaning.

3.3 System Design Summary

Summarizing the requirements so far:

- The system must perceive the DOM via snapshots, in a compressed form.
- The system must generate persistent change rules as JS code.
- The system must execute that code on the DOM.
- The system must preserve or re-apply changes on page reload.

We use tool-calling agentic system as the architectural base, as it shows significant performance boosts across a range of tasks [3] relative to LLM-driven systems without tools. Conceptually, these tools define the agent’s *action space*: the set of operations through which the model can inspect the page and commit changes.

To avoid context bloat with unnecessary information the agent must receive a drastically limited DOM snapshot initially. However, this risks losing information critical to the user’s task. The action space must therefore include a way to query a portion of the DOM in less compressed form.

Finally, the agent must be able to record new rules, which are JavaScript functions applied to the page’s elements.

This structure maps onto a three-action interface discussed in depth in the Implementation section — `get_map_of_dom`, `show_in_dom`, and `set_update_rule`.

3.4 User Testing Protocol

The goal of the trial is to assess whether the work addresses the originally stated problem:

How can adaptive generative interfaces counteract systematically misaligned business and user goals in user interfaces by re-aligning the interface with a user’s articulated goal?

More precisely — can the solution adequately solve the problem in at least a single case of a misaligned interface?

The setup is a Dia (Chromium) browser on the researcher's laptop with an extension built from the current version of code (commit `f1ae9bbfbc02c663c336e78c4772b2378ea9e77d`).

The questionnaire follows (experiment language — Russian):

1. Встречались ли вы со случаями, когда интерфейс какого-либо сайта затрудняет его использование для ваших личных целей и задач, то есть не подходил вам на сто процентов?
 1. No — end of test.
2. Что это был за сайт и как именно интерфейс вам мешал? Вспомните один любой случай.

The respondent proceeds to open the website. The researcher then opens the browser extension and instructs the user on how to use it. The respondent is invited to use the extension to customize the website. After customization, a questionnaire on the result follows:

1. Did the transformation occur? (set by the researcher)
2. Соответствует ли новая версия сайта вашим задачам больше, чем оригинальная?
3. Насколько вы оцените новую версию сайта по шкале от -3 до $+3$, где -3 — новая версия сайта сильно менее полезна, чем оригинальная, 0 — новая версия сайта не отличается от оригинальной в контексте решения личных задач, $+3$ — новая версия сайта сильно более полезна, чем оригинальная.

The testing was conducted in an informal, uncontrolled setting and cannot be used to compare different methods, models, or approaches. What it does show is the feasibility of the approach and that it can be useful for certain tasks.

3.5 AI Usage Disclosure

AI agents via Cursor Editor and Claude Code CLI were used extensively in writing the prototype code. All generated code was fully reviewed and verified manually, and all architectural decisions were made explicitly by the authors.

Chapter 4

Implementation

This section describes how the final iteration of the system works in detail. The code repository is available in the appendix.

4.1 Overview

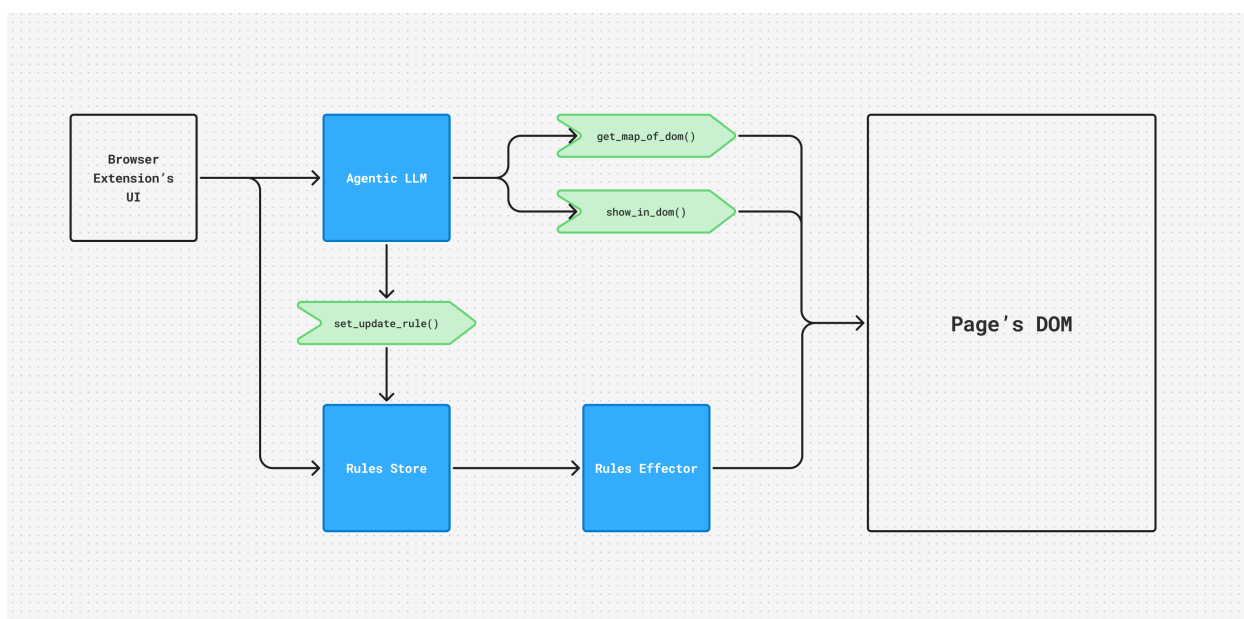


Fig. 4.1. Internet Shaper's system architecture

The browser extension can be seen as two connected systems: there is an agent with access to tools, which in turn use browser APIs and processing to get data from the page. Separately, there is a rule storage module and an engine for applying rules to web pages on load and on content change.

Note that the LLM agent does not affect the DOM directly — it only populates the rules store, which can then be managed by the user and applied by the extension.

To allow different layers of the system to communicate without friction, we built a helper framework called Fiber. It provides unified contexts for code execution, handling browser extension constraints with an Remote Procedure Call (RPC) implementation of the most useful APIs. This is described in detail in a later section.

4.2 Agentic System

We use Anthropic’s model harness with their best practices to implement tool use. The loop is: send system prompt + conversation history + tool definitions → receive content blocks → execute any `tool_use` blocks → push `tool_result` → repeat until no tool calls or `stop_reason === "end_turn"`.

4.2.1 Action Space

The agent’s action space is exposed as three tools.

`get_map_of_dom()` — returns a compact page DOM structure. It aggressively excludes elements likely to have little semantic meaning via lossy DOM compression: it filters attributes, removes high-frequency classes, collapses single-child wrapper chains, and truncates repeated sibling structures. This action gives the model an overview of the page while preserving hierarchy and broad context.

Prompt caching is applied to the system prompt and the `get_map_of_dom` result (the largest context item), reducing cost on multi-turn conversations.

`show_in_dom(query_selector, depth)` — returns HTML for a specific element from the captured DOM. The `depth` argument controls how many descendant element levels are included below the matched node; omitted `depth` defaults to three. This action is used after `get_map_of_dom()` when the model needs more local detail for selector construction or for understanding dynamic content markers. When

descendants deeper than the requested depth are omitted, they are replaced with a short comment indicating the number of hidden children.

`set_update_rule(label, query_selector, logic)` — sets a persistent update rule that will be applied to all elements matching the CSS selector every time the page loads. The `logic` parameter is JavaScript code that executes with `element` bound to each matching DOM element. The logic has no access to `window`, `document`, or any global APIs — only the `element` variable is available. The logic must be idempotent: running it on the same element multiple times must produce the same result as running it once.

The system prompt instructs the agent to prefer stable selectors (class names, `data-*` attributes, semantic tags), and to write idempotent logic that sets absolute state rather than toggling or accumulating effects.

Both `get_map_of_dom` and `show_in_dom` operate on a captured snapshot of the DOM taken at the time the user submits a request. `set_update_rule` does not immediately apply — it appends to a list of rules that are applied to the live page after the agent finishes.

4.2.2 Workflow

The agent follows a fixed exploration-then-action workflow. It must always begin by calling `get_map_of_dom` to form a structural understanding of the page before taking any other action. From this map it identifies candidate elements relevant to the user's request. When it needs full attribute and children detail for a specific element — for example, to construct a precise selector or inspect dynamic content markers — it calls `show_in_dom`.

After this exploration phase the agent emits one or more `set_update_rule` calls, each representing a target query selector and JavaScript logic to apply. The conversation loop continues until the agent produces a reply with no tool calls, or

the stop reason is `end_turn`, at which point the collected rules are applied to the live page.

Rules are saved to `localStorage` keyed by `internet-shaper-rules:${hostname}` as an array of `{ label, query_selector, logic, enabled }` objects. On every page load, `app.ts` reads the stored rules and calls `applyRules`, which runs via `executeInMainWorld` so rule logic has access to the real document. Inside the main world, rules are merged into `window.__internetShaperRules`, and a `MutationObserver` on `document.body` re-applies rules to any newly added nodes. This is done to handle single-page applications (SPAs) like YouTube and Instagram that load content dynamically without full page reloads.

Example. Rule for `www.google.com`: On `.YzCcne` query selector Apply `element.style.display = 'none'`

4.2.3 Proprietary model aside

The model of choice for the prototype is Anthropic's Claude Sonnet 4.6, one of the state-of-the-art models at the time of writing.

As per the [4]'s recommendation, we see the need to justify this choice. Because we are not testing the models themselves, but rather a novel application that utilizes LLMs for part of its functionality, there is little risk of non-reproducibility. It is reasonable to expect that future models will surpass the capabilities of today's closed-source state-of-the-art models, which would also improve the prototype's performance.

As a limitation however, we will not compare the performance of the prototype across different models. That would amount to testing the models themselves, which would not be reproducible with closed-source models.

4.3 DOM Compression

Following the system design, the current system uses two levels of DOM reduction. First, the browser extension captures a visibility-filtered DOM snapshot. Second, `get_map_of_dom` turns that snapshot into a lossy structural map. The purpose is not to identify only the task-relevant elements, but to preserve as much broad page context as possible while making the page small enough for the LLM to process.

4.3.1 Visibility filtering

At request time, the extension captures `document.documentElement.outerHTML` after removing body subtrees that are not visible according to computed style: descendants of `display: none`, elements with `visibility: hidden`, and elements with zero opacity. The heuristic is that users generally request changes to what they can see. Unlike the lossy structural map, this capture-time filtering does not insert placeholders for the removed invisible subtrees.

4.3.2 Lossy structural map

`get_map_of_dom` then produces a compacted structural map of the captured DOM, used as the agent’s initial view. Each aggressive compression rule follows a specific heuristic.

1. Strip most attributes — only attributes likely to help identify elements or preserve meaning are kept: `alt`, `aria-label`, `class`, `id`, `label`, `name`, `placeholder`, `role`, `title`, `type`, `value`, and `data-*`. Inline styles, `src`, `href`, and other attributes are removed.
2. Remove high-frequency classes — classes appearing on more than 5% of all elements are dropped, as they are likely design-system or framework classes with little semantic meaning. We experimented with this common-class threshold. Thresholds above 30% flagged almost no classes on the DOMs we tried, while 20% provided a reasonable baseline. The current 5% threshold also worked empirically, but the relationship between this threshold and system performance requires more systematic evaluation.

3. Collapse single-child wrapper chains — non-root elements with no siblings and exactly one child are collapsed. The removed wrappers' attributes are merged into the surviving element where possible, and a short comment such as `<!-- -N wrappers -->` is inserted. DOMs are full of structural single-child `div` tags that are often artifacts of component frameworks rather than semantically meaningful content.
4. De-duplicate repeated sibling structures — consecutive sibling nodes with similar tag, id, class-token sets, and immediate child-tag sequences are compacted by keeping the first element and replacing the rest with a short comment such as `<!-- -N siblings -->`. This handles lists of content with similar structure while preserving the fact that a repeated structure existed.
5. Normalize whitespace — insignificant whitespace-only text nodes are removed, and textual whitespace is collapsed outside raw-text contexts such as `script`, `style`, `pre`, and `textarea`.

These comments make the compressed DOM more interpretable because they expose where structure was removed. They also consume part of the final context: in the 75-snapshot evaluation described below, comments occupied 6.5% of compressed output characters on average and 5.9% at the median. This overhead is acceptable for now, but future work should evaluate whether shorter placeholders preserve enough structural information at lower token cost.

4.4 Browser Extension Wrapper (Fiber)

We have discussed extensively how the browser extension is a natural fit for this system: it gets unique read/write access to any page's DOM and JavaScript runtime without requiring any cooperation from the page's authors.

The challenge however, is that browser extensions operate across multiple execution contexts that cannot directly call each other: content scripts (isolated world), the page's JavaScript runtime (main world), and the background service worker. A

single user-visible action — like applying a rule — requires jumping across all three. In practice this means constantly passing events between contexts within a single logical flow.

To manage this, and improve development experience, we built Fiber — a small, reusable extension framework that abstracts the context-switching problem. Fiber wraps Chrome APIs in an `ext` proxy: calling `ext.scripting.executeScript()` from a content script feels like a direct call, but under the hood Fiber serializes it as a typed RPC message (`{ id, method, args }`) over `chrome.runtime.sendMessage` to the background service worker, which then executes the actual Chrome API and returns `{ id, result | error }`. The most important case is `ext.scripting.executeInMainWorld(fn, args)`, which serializes the function to a string before sending, then reconstructs and evaluates it in the page's main world using Trusted Types to satisfy the page's Content Security Policy.

4.5 Security Concerns

The prototype in its current state has several vulnerabilities and must not be used in a public setting:

- It stores API keys in the browser's storage in plain, un-encoded form.
- It has no protection against prompt injections that can be present in page content. This can enable severely harmful behavior up to remote code execution, as the rule application sandbox can fetch data from any URL.

Chapter 5

Results

5.1 Browser Extension

5.2 User Testing

5.3 DOM Compression

We evaluated DOM compression on 75 snapshots collected from high-traffic websites. The sampling pipeline started from a public list of common domains, removed login- or payment-gated services, adult sites, bot-blocked pages, legal boilerplate pages, regional duplicates, and broken services, then sampled up to three additional same-domain pages per seed domain. For each retained page we saved the raw HTML, a visibility-filtered HTML snapshot, and the compressed `get_map_of_dom` output.

TABLE 5.1
DOM compression summary

Metric	Mean	Median	Min	Max
Raw HTML characters	937,528	735,180	50,097	4,411,456
Visible HTML characters	417,214	289,292	21,212	2,353,194
Compressed characters	38,703	24,371	1,814	333,411
Comment characters	1,988	1,287	161	9,466
Raw HTML tokens	334,511	240,987	18,490	1,338,122

Metric	Mean	Median	Min	Max
Visible HTML tokens	160,711	109,063	7,188	1,028,594
Compressed tokens	12,150	8,041	507	108,008

Visibility filtering roughly halved the token count: the median visible snapshot retained 46.3% of raw tokens. The lossy structural map then reduced the median visible snapshot to 7.7% of its visible-token size, or 2.9% of the original raw-token size. The largest raw snapshot in this evaluation exceeded 1.3 million tokens, while the largest compressed map was 108,008 tokens, within the context budgets of the models used by the prototype. These results suggest that the compression pipeline is sufficient for feasibility, while still leaving room for optimization on very large or unusually repetitive pages.

The placeholder comments inserted by lossy compression are not free: they occupied 6.5% of compressed output characters on average and 5.9% at the median. They make the map easier to interpret by marking where wrappers, siblings, or deeper children were removed, but they should be treated as part of the compression design rather than ignored overhead.

Chapter 6

Discussion

6.1 Emergent Behaviors

Even more complex queries produced unexpectedly positive results, with the agent exhibiting emergent behavior that was not planned for:

- When asked to “add a cat to the page”, it called an external API dedicated to serving a random cat image and injected a new image element into the page. > Cat image added — a random cat image (fetched from cataas.com) has been inserted into the center of the page, with rounded corners and a soft shadow.
- For requests more complex than “remove element A”, the agent immediately used variables, conditionals, and accessed properties of child nodes from the parent.
- When asked to turn a list of articles on Substack into a grid, it produced a set of approximately seven rules, adapting not only the layout but also the cards themselves — their headers, thumbnails, and action menus — all to better fit the new grid context. This example shows the agent’s high-level understanding of tasks rather than mechanical rule application.
- When asked to restructure the page in a major way, moving sections around, the agent created a single rule for the main element containing all the logic, rather than many individual rules.

Chapter 7

Conclusion

7.1 Limitations

1. The LLM is provided a single screen in an application and is not aware of the multi-screen context. The risk is that it might produce adaptations incompatible with the overall flow on more complex tasks.
2. The system can only influence a single screen, making construction of multi-step flows and app-wide modifications difficult.
3. Rules are applied to all pages that match the domain; no URL path matching occurs. As a result, rules sometimes unintentionally apply to pages they were not intended for. This was an architectural decision that helps handle dynamically constructed pages (e.g. `domain.com/user/<id>`). Future work can investigate the feasibility of requiring a URL mask for each rule, or a similar approach to limit unexpected rule applications.
4. The LLM has access only to data visible to the user and present in the DOM. For a subset of interactions — specifically those involving dynamically loaded data — access to the underlying JavaScript data structures could be helpful. The same applies to API actions, of which the model has no context at all.

Bibliography cited

- [1] L. Ning *et al.*, “A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models.” Accessed: Apr. 14, 2026. [Online]. Available: <http://arxiv.org/abs/2503.23350>
- [2] J. Zhang, K. Chen, Z. Lu, E. Zhou, Q. Yu, and J. Zhang, “Prune4Web: DOM Tree Pruning Programming for Web Agent.” Accessed: May 18, 2026. [Online]. Available: <https://arxiv.org/abs/2511.21398>
- [3] “Tool use with Claude.” Accessed: Apr. 20, 2026. [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>
- [4] A. Palmer, N. A. Smith, and A. Spirling, “Using proprietary language models in academic research requires explicit justification,” *Nature Computational Science*, vol. 4, no. 1, pp. 2–3, Jan. 2024, doi: 10.1038/s43588-023-00585-1.

Appendix A

Source Code

WIP